

DAGSmith: Dependency-Aware Rewriting for dbt-Style SQL Pipelines

Jie Liu

jiezzliu@umich.edu

Computer Science and Engineering
University of Michigan, Ann Arbor
USA

Lin Ma

linmacse@umich.edu

Computer Science and Engineering
University of Michigan, Ann Arbor
USA

Barzan Mozafari

mozafari@umich.edu

Computer Science and Engineering
University of Michigan, Ann Arbor
USA

ABSTRACT

Modern analytics is shifting from isolated, human-written SQL queries to large, recurring SQL programs produced and maintained by data teams, BI tools, and code-generation assistants. These programs increasingly appear as explicit dependency graphs of models, tasks, assets, and materialized tables. The graph makes inter-query dependencies visible, but database optimizers and most query rewriters still treat each SQL statement as the unit of optimization. This creates optimization opportunities that are invisible from inside one query: cross-boundary semantic reasoning, intermediate contract freedom, non-local semantic reuse, rewrite/materialization coupling, freshness-aware recomputation, and global composition of interacting rewrites.

We introduce DAGSmith, a system for dependency-aware rewriting of dbt-style SQL pipelines. DAGSmith treats a pipeline as a stable SQL pipeline DAG whose node definitions, dependency edges, output contracts, materialization choices, and refresh requirements are jointly available. It combines structural analysis over the DAG, LLM-guided refactoring, warehouse-backed equivalence checks, learned cost estimation, materialization tuning, frequency-aware scheduling, and global rewrite selection. DAGSmith rewrites the structure around individual queries: factoring non-local shared work, pruning unused or mistimed computation, aligning refresh work with data-change and consumption rates, and selecting a globally consistent set of changes. This paper defines the optimization setting, identifies the opportunities created by explicit dependencies, and presents the architecture of DAGSmith.

1 INTRODUCTION

Modern analytics is shifting from human-written, isolated SQL queries to large, continuously maintained SQL programs. Data teams are asked to serve more stakeholders, combine more sources, support AI-facing datasets, and expose governed data products faster; at the same time, BI tools, analytics engineering frameworks, and code-generation assistants increasingly generate or maintain SQL on behalf of users. dbt Labs reports that 70% of analytics professionals already use AI for code development, 57% still spend most workdays maintaining or organizing datasets, and nearly 65% believe enabling nontechnical users to create transformed and governed datasets would improve organizational efficiency [?]. The consequence is an efficiency gap: organizations and automated tools produce increasingly large, repetitive, and semantically rich SQL pipelines, while database optimizers still see mostly one submitted statement at a time.

Query rewriting – Query rewriting has become an important way to close this gap. Rule-based, synthesis-based, and LLM-assisted

rewriters such as *SlabCity* [?], *GenRewrite* [?], *WeTune* [?], and *R-Bot* [?] improve SQL before it reaches the database optimizer. However, these systems inherit the same unit of optimization as the optimizer: one query. They cannot generally ask why a relation is being computed, which later outputs depend on it, whether sibling computations are semantically redundant, or whether expensive work is being refreshed more often than its inputs change.

Explicit pipeline structure – At the same time, modern analytics pipelines increasingly make dependencies explicit. Analytics engineers now write and maintain transformation programs in which one SQL-defined asset consumes another, producing a source-level graph of recurring computation.¹ We view these as instances of a *SQL pipeline DAG*: a recurring directed acyclic graph whose nodes are SQL transformations or materialized relational assets, and whose edges are data dependencies. Figure 1 shows the kind of structure such a DAG exposes: dependencies, intermediate boundaries, refresh cadences, and downstream consumers are visible at the source level rather than hidden inside a single query.

Limitation of existing approaches – Existing database techniques use only fragments of this information. Single-query rewriting optimizes one statement at a time. Multi-query optimization shares work across a workload, but typically treats the workload as a flat set of queries and relies on syntactic overlap [?]. Materialized-view selection chooses useful persisted expressions, but does not generally rewrite the maintained pipeline around dependency structure, intermediate contracts, or refresh schedules [?]. Workflow schedulers know dependencies, but usually optimize execution order, invalidation, or retry policy rather than semantic SQL. This comparison belongs early because it identifies the missing optimization object: the maintained SQL pipeline program itself.

Key insight – The key observation behind DAGSmith is that dbt-style SQL pipeline DAGs expose six optimization opportunities that are unavailable to a single-query rewriter, a flat multi-query optimizer, or a materialized-view advisor. **(1) Cross-boundary semantics**: the optimizer can reason about the meaning of a node by reading related upstream, sibling, and downstream SQL definitions. **(2) Intermediate contract freedom**: internal nodes can be narrowed, split, merged, or removed as long as externally consumed outputs are preserved. **(3) Non-local semantic reuse**: logically equivalent work may appear in different branches with different syntax, names, predicates, or aggregation boundaries. **(4) Rewrite-materialization coupling**: whether a rewrite helps depends on table/view/incremental materialization choices, and those choices

¹Examples include dbt [?], SQLMesh [?], Dagster [?], Airflow [?], Prefect [?], Argo Workflows [?], Databricks Workflows [?], and medallion lakehouse pipelines [?].

depend on the rewritten graph. **(5) Freshness-aware recomputation:** in cloud warehouses, cost and latency depend on how often sources change, how often outputs are consumed, and which nodes lie on critical refresh paths. **(6) Global rewrite composition:** DAG-level rewrites can overlap or interact, so locally beneficial changes must be selected as a compatible set.

System overview – DAGSmith exploits these opportunities with a dependency-aware rewriting pipeline. It identifies promising regions of the SQL pipeline DAG, asks an LLM to propose graph-level refactorings, filters unsafe or counterproductive proposals, verifies adopted rewrites with warehouse-backed diagnostics, evaluates each candidate under an optimized materialization configuration, and selects a non-conflicting set of rewrites globally. The result is a rewritten pipeline that remains executable by the same warehouse and orchestration framework, but exposes less redundant work and better aligns computation with materialization and refresh behavior.

Contributions – This paper makes the following contributions:

- We introduce, to the best of our knowledge, the first holistic source-to-source query rewriting framework for dbt-style SQL pipeline DAGs: recurring SQL transformation pipelines with explicit dependencies, intermediate node contracts, materialization choices, and refresh constraints.
- We identify six optimization opportunities enabled by explicit pipeline dependencies: cross-boundary semantics, intermediate contract freedom, non-local semantic reuse, rewrite-materialization coupling, freshness-aware recomputation, and global rewrite composition.
- We present DAGSmith, which combines structural graph analysis, LLM-guided refactoring, empirical equivalence checking, learned cost modeling, materialization tuning, frequency-aware scheduling, and global candidate selection.
- Our evaluation will show that DAGSmith reduces end-to-end warehouse cost by **XX%** over the original pipeline, by **YY%** over materialization-only tuning, and by **ZZ%** over single-query rewriting baselines, while improving sink refresh latency by **LL%**. These placeholders will be replaced with final measurements in Section 4.

The rest of the paper is organized as follows. Section 2 defines the setting and illustrates the opportunity space. Section 3 presents the design of DAGSmith. Section 4 outlines the evaluation plan, Section 5 situates the system relative to prior work, and Section 6 concludes.

2 OPPORTUNITIES IN SQL PIPELINE DAGS

Before presenting our approach, we describe the optimization setting that DAGSmith targets and illustrate, through three examples drawn from or inspired by real-world analytics pipelines, the kinds of dependency-aware optimizations that this setting enables but that fall outside the reach of existing techniques.

2.1 Setting

A *SQL pipeline DAG* is a directed acyclic graph $G = (V, E)$ in which each node is a relational transformation or a persisted relational asset, and each edge records that one node consumes the output of another. A node may be implemented as a SQL model, a notebook

cell that issues SQL, a warehouse task, or a materialized table maintained by an orchestration framework. The graph may be declared directly, as in Airflow [?] and Argo [?], inferred from dataflow, as in Prefect [?], or induced by model references, as in dbt [?] and SQLMesh [?]. In asset-oriented systems such as Dagster [?], the same idea appears as an asset graph: nodes are persistent assets and edges describe how one asset is derived from another.

dbt is the running example in this paper because its SQL model boundaries and `ref()` dependencies make the setting especially explicit. However, the optimization problem is broader. Databricks Workflows [?] and Lakeflow Jobs [?] expose task dependencies as a job DAG; Databricks’ medallion architecture [?] encourages multi-hop transformations from bronze to silver to gold tables; Luigi [?] and the systems above all represent recurring data work through dependency graphs. DAGSmith targets the subset of these pipelines where the node logic is available as SQL, or can be compiled to SQL, so that the optimizer can reason about relational semantics rather than only task ordering.

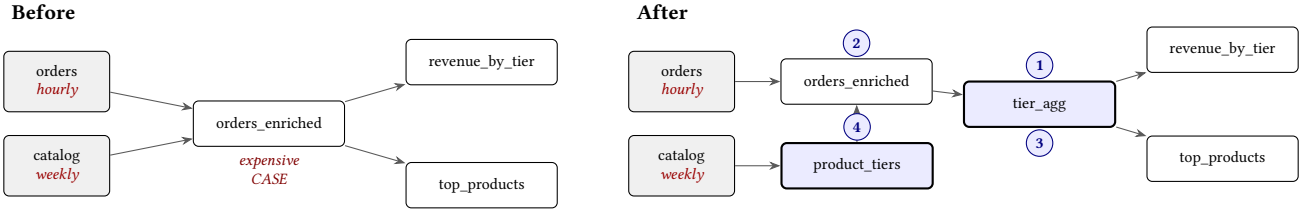
This setting differs from traditional query optimization in six ways that matter for our problem. Figure 1 previews several of the changes these differences make possible.

(1) Cross-boundary semantics. The optimizer can interpret a node by reading the SQL definitions around it. Parent, sibling, and downstream nodes often explain why a relation exists, which columns matter, and whether a dependency is semantically necessary. This is stronger than expanding a view inside one query: the optimizer reasons over maintained source definitions and their relationships, not just over a single compiled statement.

(2) Intermediate contract freedom. Not every node in the DAG has the same semantic status. Sinks consumed by dashboards, downstream pipelines, applications, or ML features are externally visible contracts. Many intermediate nodes are implementation details. A dependency-aware optimizer can therefore preserve the outputs that matter while changing intermediate schemas, boundaries, and materializations. A single-query rewriter normally cannot make this distinction because the query boundary itself is the contract.

(3) Non-local semantic reuse. The DAG is a maintained software artifact. Teams version it, review it, test it, document it, and execute it repeatedly. This stability makes source-to-source optimization worthwhile: expensive analysis can be amortized over many recurring executions, and semantically redundant work can be found across distant branches rather than only inside one query text. This is especially relevant as projects grow. dbt Labs notes that what counted as a large project has grown from roughly 150 models to 500, then 1,000, and that an appreciable fraction of users now work in projects with 5,000 models or more [?]. dbt’s own internal analytics project reportedly processes roughly 1,500 models across multiple jobs and schedules [?].

(4) Rewrite-materialization coupling. Whether a node should be a table, view, incremental model, or ephemeral expression depends on its parents, descendants, fanout, refresh behavior, and the refactorings under consideration. A shared intermediate may be valuable as a table and harmful as a view; a narrow projection may make an upstream table unnecessary; merging two nodes may reduce



- (1) *Cross-boundary semantics.* The optimizer can interpret a node by reading the SQL definitions around it, not just the node's own query text.
- (2) *Intermediate contract freedom.* The optimizer can narrow, split, merge, or remove internal nodes as long as externally visible outputs are preserved.
- (3) *Rewrite-materialization coupling.* A shared intermediate is useful only under the right materialization choice; graph rewriting and physicalization must be optimized together.
- (4) *Freshness-aware recomputation.* The expensive classification depends only on weekly data but lives inside an hourly model; splitting it onto the slow cadence requires a DAG rewrite invisible to per-query cost.

Figure 1: A compact example of the optimization opportunities exposed by an explicit SQL pipeline DAG. Once dependencies are visible, the optimizer can reason across node boundaries, rewrite internal contracts, couple refactoring with materialization, and split slow-changing logic off fast refresh paths.

writes but increase downstream recomputation. Materialization is therefore not a local property of one model.

(5) **Freshness-aware recomputation.** Different sources update at different rates, and different downstream consumers require fresh results at different rates. A model's useful execution frequency is shaped by both upstream data-change rates and downstream demand. In cloud warehouses, these rates translate directly into cost and latency objectives: if a weekly source feeds an hourly model, any computation that depends only on the weekly source is being repeated unnecessarily; if an output is consumed daily, hourly refresh may be waste even when the query itself is individually optimized.

(6) **Global rewrite composition.** DAG-level rewrites interact. Two local changes may touch the same node, make incompatible assumptions about an intermediate contract, or change the materialization choice that makes another rewrite profitable. A dependency-aware optimizer therefore needs a global composition step that selects a compatible set of rewrites rather than greedily applying every local improvement.

Together, these properties define a distinct optimization setting. The optimizer is reasoning not about one query, nor merely about a set of independent queries, but about a stable SQL pipeline program whose graph structure, intermediate contracts, materialization choices, and refresh behavior interact.

2.2 Opportunity Taxonomy

The properties above create a broader opportunity space than the three examples in this section can cover exhaustively. We organize the space into six recurring classes that map directly to the components of DAGSmith in Section 3.

Cross-boundary semantics. The optimizer can read related node definitions to discover semantic facts that are not present in the current SQL file. This enables rewrites such as replacing a dependency on a sibling node with an equivalent predicate over a shared parent.

Intermediate contract freedom. Intermediate models often exist for developer organization rather than as externally visible contracts. If the final outputs are preserved, the optimizer may narrow

schemas, remove unused columns, change intermediate grouping levels, or replace an intermediate with a cheaper equivalent.

Non-local semantic reuse. Multiple branches may compute the same logical quantity using different syntax, predicates, joins, or naming conventions. The opportunity is to factor the shared computation into a new model or asset, then rewrite consumers to use it.

Rewrite-materialization coupling. The best structure depends on which nodes are persisted. A rewrite that creates a shared node only helps if that node is materialized appropriately; inlining it as a view may simply duplicate the work downstream.

Freshness-aware recomputation. The cost of a node is not just its per-run cost. It is the cost multiplied by how often the node must produce useful new output. The optimizer can skip over-scheduled work, move slow-changing logic onto a slow cadence, or avoid forcing a slow branch to refresh at the rate of a fast one.

Global rewrite composition. Local wins can conflict. Two refactorings may touch the same model, make incompatible assumptions about an intermediate contract, or interact through materialization choices. A dependency-aware optimizer needs a global selection step rather than a greedy sequence of local rewrites.

We now illustrate three representative classes that DAGSmith discovers. Each requires the optimizer to read and reason about SQL definitions *across* model boundaries within the DAG. The SQL in each example is simplified for presentation; domain-specific details are abstracted away.

Example 1: Non-local semantic reuse. A pipeline contains seven classifier models that each tag orders with a service category (dialysis, psychiatric, and five others). The input is a `claims` table in which each row represents one item of an order, so a single order has many rows. Each classifier scans `claims`, keeps rows whose codes match its category's code list, and returns the distinct `order_ids` of those rows. The seven classifiers have *different* SQL: different predicates, different code vocabularies, and in some cases different join structures. Two representative classifiers are shown on the left of Figure 2.

The seven classifiers share no common subexpression, yet they all perform the same *semantic operation*: for each order, decide

Before: 7 classifiers (2 shown)

```

1 -- dialysis_classifier
2 SELECT DISTINCT order_id
3 FROM {{ ref('claims') }}
4 WHERE bill_code LIKE '72%'
5 OR taxonomy IN (set_dia_tax)
6 OR diagnosis IN (set_dia_dx)
7
8 -- psychiatric_classifier
9 SELECT DISTINCT order_id
10 FROM {{ ref('claims') }}
11 WHERE revenue IN (set_psy_rev)
12 OR diagnosis BETWEEN
13 '290' AND '319'
14

```

After: precomputation+lookups

```

1 -- order_flags (new)
2 SELECT order_id,
3 MAX(CASE WHEN bill_code LIKE '72%'
4 OR taxonomy IN (set_dia_tax)
5 OR diagnosis IN (set_dia_dx)
6 THEN 1 ELSE 0 END) AS f_dia,
7 MAX(CASE WHEN ...
8 THEN 1 ELSE 0 END) AS f_psy,
9 ... -- 7 categories
10 FROM {{ ref('claims') }}
11 GROUP BY order_id
12
13 -- dialysis_classifier (rewritten)
14 SELECT order_id
15 FROM {{ ref('order_flags') }}
16 WHERE f_dia = 1

```

Figure 2: Cross-model semantic factorization. Seven classifiers with no syntactic overlap perform the same semantic operation on a shared parent. DAGSmith synthesizes a new shared model that pre-aggregates all classifier flags in one pass.

whether any of its rows match a category-specific predicate. DAGSmith recognizes this cross-model pattern and factors it. It synthesizes a new intermediate model `order_flags` that, in a single pass over `claims`, computes one boolean flag per category by aggregating each category’s predicate with `MAX(CASE WHEN ... THEN 1 ELSE 0 END)` grouped by `order_id`. Each original classifier is then rewritten as a lightweight lookup: select the orders whose corresponding flag in `order_flags` is 1. Seven full scans and seven independent grouping passes over `claims` are replaced with one pre-aggregation pass and seven inexpensive boolean filters.

This optimization requires three capabilities that go beyond traditional techniques. First, the optimizer must recognize that seven syntactically dissimilar models perform the same semantic operation, which syntactic subexpression matching, the basis of multi-query optimization (MQO) [?] and common subexpression elimination, cannot do. Second, it must *invent* a new intermediate model absent from the original project; materialized view recommendation tools [?] synthesize candidate views, but they too rely on syntactic subexpression mining, of which the seven classifiers share none. Third, it must rewrite the dependent models to consume the new intermediate while preserving end-to-end equivalence, a multi-model restructuring that single-query rewrite systems [?] do not attempt.

Example 2: Cross-boundary semantics and contract freedom. The same project contains a classifier `nursing` that filters for skilled-nursing claims by facility code, then performs an anti-join against a sibling model `equipment` to exclude equipment-related claims (Figure 3, left). The anti-join forces the engine to materialize `equipment`, build a hash table, and probe it for every row of `claims`; it also introduces a DAG dependency that serializes the two models.

By reading the SQL definition of `equipment`, DAGSmith discovers that it computes nothing more than a single-predicate filter on a shared parent. The anti-join is therefore equivalent to an inline scalar predicate on `claims`, as shown on the right of the figure: the hash anti-join, the auxiliary scan, and the cross-model DAG dependency are all eliminated. The same reasoning generalizes to every sibling classifier that uses `equipment` as an exclusion list, compounding the benefit across the pipeline.

Before: anti-join

```

1 -- nursing
2 SELECT *
3 FROM {{ ref('claims') }} c
4 WHERE c.facility_code
5 IN ('31','32')
6 AND NOT EXISTS (
7 SELECT 1
8 FROM {{ ref('equipment') }} e
9 WHERE e.order_id = c.order_id
10 AND e.line_id = c.line_id)
11
12 -- equipment (just a filter)
13 SELECT *
14 FROM {{ ref('claims') }}
15 WHERE category = 'equipment'

```

After: predicate

```

1 -- nursing (rewritten)
2 SELECT *
3 FROM {{ ref('claims') }} c
4 WHERE c.facility_code
5 IN ('31','32')
6 AND (c.category IS NULL
7 OR c.category <> 'equipment')
8
9
10 -- equipment is removed if no
11 -- other model depends on it;
12 -- otherwise its definition
13 -- is unchanged.
14

```

Figure 3: Cross-boundary semantic reasoning. By reading the SQL of the sibling `equipment` model, DAGSmith determines that the anti-join is equivalent to a scalar predicate on a shared parent. The hash anti-join, the auxiliary scan, and the DAG dependency are all eliminated.

The equivalence enabling this rewrite is not declared anywhere; it is *embedded in the SQL of another model file*. A traditional engine sees `equipment` either as a base table to be probed or as a view to be expanded into the local query, neither of which performs the cross-model analysis needed to eliminate the join entirely. We acknowledge a tradeoff: the rewrite couples `nursing` to the current definition of `equipment`, so a future change to that definition would require updating the rewritten consumers. In practice, stable upstream models such as `equipment` change rarely, and DAGSmith re-evaluates the project after each developer change, so the cost of re-deriving the rewrite is amortized over many executions.

Example 3: Freshness-aware recomputation. Different sources in a SQL pipeline DAG update at different rates, and the structure of the DAG determines how much computation must be repeated at each high-frequency refresh. Consider a model that joins fast-changing transactional data with slow-changing reference data and applies expensive classification logic to the slow-changing side (Figure 4, left).

Before: hourly classification

```

1 -- order_summary (hourly)
2 SELECT o.order_id, o.amount,
3 CASE
4 WHEN p.code IN (set_A) THEN 'A'
5 WHEN p.code IN (set_B) THEN 'B'
6 ...
7 END AS category
8 FROM {{ ref('orders') }} o
9 JOIN {{ ref('catalog') }} p
10 ON o.product_id = p.product_id
11
12 -- orders: hourly
13 -- catalog: weekly

```

After: split by frequency

```

1 -- product_categories (weekly)
2 SELECT product_id,
3 CASE
4 WHEN code IN (set_A) THEN 'A'
5 WHEN code IN (set_B) THEN 'B'
6 ...
7 END AS category
8 FROM {{ ref('catalog') }}
9
10 -- order_summary (hourly)
11 SELECT o.order_id, o.amount,
12 pc.category
13 FROM {{ ref('orders') }} o
14 JOIN {{ ref('product_categories') }} pc
15 USING (product_id)

```

Figure 4: Frequency-aware restructuring. Splitting the model along the frequency boundary moves expensive classification onto a weekly cadence; the hourly model becomes a lightweight join.

Because `orders` changes hourly, `order_summary` re-executes hourly, and the multi-branch classification over `catalog` is recomputed on every hourly refresh, despite its inputs changing only

weekly. DAGSmith traces the update frequency from each source through the DAG, identifies that this model has *mixed-frequency inputs*, and splits it along the frequency boundary. The expensive classification is extracted into a new slow-frequency model, and the original model becomes a lightweight equi-join. When this pattern recurs across many consumers of the same slow-changing reference data, the savings compound: each hourly refresh avoids redundant recomputation of logic whose inputs have not changed.

This optimization is invisible to optimizers that treat cost as a fixed per-query quantity: the benefit of splitting the model is realized only when the two halves are known to execute at different rates. Materialized view selection considers update cost but does not restructure the DAG to align computation with update frequency, and traditional query rewriters have no notion of refresh schedule at all.

2.3 Why Existing Techniques Fall Short

The three examples above are not isolated curiosities; they illustrate a general capability gap. Single-query rewrite systems [?] operate within the boundary of one query and cannot restructure across model files. Multi-query optimization and common subexpression elimination [?] share intermediate computations across queries but rely on syntactic matching, so they miss the cross-model semantic equivalence in Example 1. Materialized view selection [?] chooses among existing view candidates and treats update cost as a static parameter rather than as a structural driver of DAG design (Example 3). Code clone detection and lineage tools can identify structural similarity or trace data provenance, but they do not perform the optimization itself.

The gap is not that any one of these techniques is poorly designed; it is that none of them targets the setting we are concerned with. Optimizing a SQL pipeline DAG requires an optimizer that can combine the six capabilities above: read semantics across node boundaries, exploit freedom over internal contracts, find semantic reuse beyond syntactic overlap, evaluate rewrites together with materialization choices, account for recurring refresh behavior, and compose interacting rewrites globally. The remainder of this paper presents an approach that integrates these capabilities into a coherent optimization framework.

3 APPROACH

DAGSmith optimizes a SQL pipeline DAG along three interacting dimensions: graph refactoring, materialization, and refresh frequency. The benefit of a refactoring depends on which nodes will be materialized, the optimal materialization configuration depends on the structure of the DAG, and both depend on how often each node executes. A pipeline with hundreds or thousands of models admits an enormous space of candidate refactorings, materialization assignments, and refresh schedules; exhaustive search is infeasible, and the three dimensions cannot be optimized in independent passes without missing interactions or producing decisions that conflict at the boundaries.

DAGSmith addresses this challenge with a three-stage pipeline. The first stage *identifies* promising refactoring opportunities. Structural analyses of the pipeline’s SQL rank model groups by a fingerprint-based score that estimates the potential benefit of joint refactoring;

the top-ranked groups are then passed to an LLM, which proposes one or more concrete refactorings for each. Together, the structural analyses and the LLM constitute the candidate generator: the structural analyses decide *where* in the DAG to look, and the LLM decides *what* refactoring to perform. The second stage *evaluates* each candidate refactoring under its own optimal materialization configuration. A learned cost model predicts slot time for any DAG configuration, and an iterative local linearization procedure uses these predictions to find the materialization assignment that minimizes total cost for the candidate. The third stage *composes* the surviving candidates into a single optimized DAG. An ILP selects the best non-conflicting subset of refactorings, after which a final materialization pass tunes the combined DAG. The frequency dimension contributes to the pipeline at three levels: it weights the cost-model-driven decisions during materialization tuning and refactoring selection, it corrects over-scheduled refresh rates so that nodes run only as often as their inputs change, and it contributes a class of structural refactorings in the first stage, where the LLM identifies nodes that can be split along frequency boundaries.

The remainder of this section presents the components of DAGSmith in the order they appear in the pipeline. Section 3.1 describes the structural analyses that identify candidate model groups. Section 3.2 describes how the LLM produces concrete refactorings from the resulting subgraphs. Section 3.3 describes the cost model and the iterative local linearization procedure that finds the best materialization configuration for any given DAG structure. Section 3.4 presents the ILP that selects the best non-conflicting combination of refactorings. Section 3.5 extends the pipeline with frequency-aware weighting at the materialization and selection stages and introduces the frequency-driven splitting refactoring.

3.1 Candidate Identification

The first stage of the pipeline identifies regions of the DAG where a refactoring is likely to be worthwhile. A pipeline with hundreds of SQL nodes admits an enormous number of conceivable refactorings, so applying an LLM to every node group is prohibitively expensive and produces a high rate of trivial proposals. DAGSmith addresses this with a division of labor: structural analyses over the pipeline’s SQL surface candidate regions whose dataflow exhibits patterns associated with refactoring opportunities, and the LLM examines each region and decides what refactoring, if any, is appropriate. Structural analysis is cheap and is applied to every node in the DAG; LLM reasoning is expensive and is reserved for regions where structural evidence suggests it is likely to pay off.

Dataflow representation. Each node’s SQL is compiled into a per-node dataflow representation that captures column derivations, joins, and aggregations as a tree of typed operators, with column references resolved across dependency boundaries. In dbt-style projects, these boundaries are expressed through the `ref()` graph. Consider three models:

```
1 A: SELECT k1, k2, SUM(c1) AS a1 FROM {{ ref('s1') }} GROUP BY k1, k2
2 B: SELECT k1, SUM(a1) AS b1 FROM {{ ref('A') }} GROUP BY k1
```

Model C performs both aggregations within a single SQL statement. At the SQL level, B’s reference to A is opaque: the column a1 is

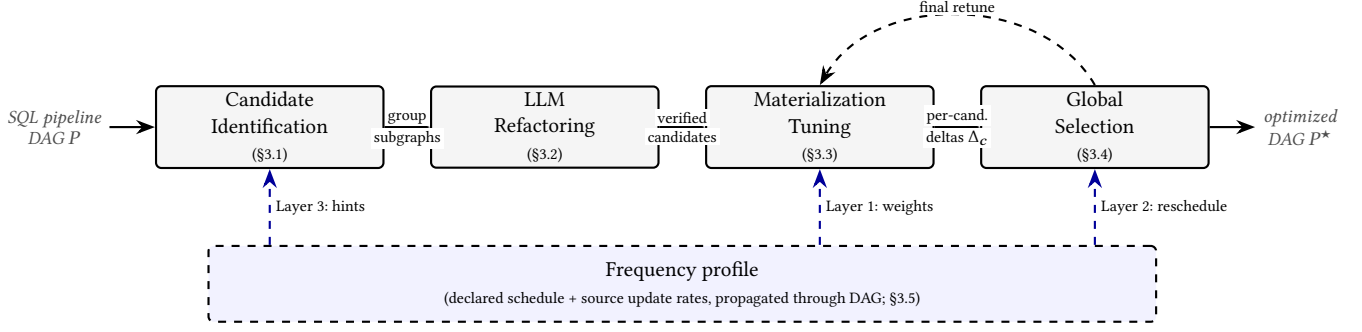


Figure 5: End-to-end pipeline of DAGSmith. The top row shows the four main stages and the artifacts that flow between them; after global selection, a final materialization pass retunes the combined DAG. The dashed band underneath is the frequency-aware overlay (§3.5): a single frequency profile feeds three of the four stages, contributing structural splitting hints, cost weights for materialization tuning, and schedule corrections at selection.

just a name produced by some upstream model. The dataflow representation resolves this reference, links the column in B back to its definition in A, and reconstructs the chain:

```
1 s1.c1 -> SUM by (k1, k2) -> SUM by k1 -> B's output
```

The same chain is contained inside C. The structural correspondence between the two-model composition and the single model is therefore visible in the representation, even though it is hidden by the `ref()` boundary in the source SQL. The analyses described below operate on this representation, which is what allows them to recognize cross-model patterns that are invisible at the SQL level.

Reuse analysis. Reuse analysis identifies groups of models that perform the same expensive operation over related inputs. From each model’s dataflow representation, it extracts a structural fingerprint summarizing the model’s expensive operations, joins and group-by aggregations, together with their anchor columns (join keys and grouping keys) and input relation lineage. Models whose fingerprints share substantial structure are grouped into candidate sets. Each candidate set is scored by an estimate of joint refactoring potential, which weights the shared operation by its expected cost and by the number of models that share it.

A high reuse score is a *targeting signal*, not an instruction to materialize a shared subexpression. Reuse analysis subsumes the syntactic shared-subexpression case targeted by multi-query optimization and materialized view recommendation, but extends it to semantically equivalent operations that are syntactically dissimilar.

Prune analysis. Prune analysis targets joins, in two forms. The first is *redundant joins*: joins whose results are not consumed by any downstream model. These can be eliminated entirely once column lineage from leaf models confirms that no downstream consumer reads from the joined output. Semi-joins fall into this category when the filtering set is statically derivable from columns already present in the probed relation, in which case the join itself can be replaced with a predicate. The second form is *mistimed joins*: joins that are correct in semantics but are computed too soon. When one side of a join feeds into expensive downstream operations whose results do not depend on the other side’s columns, the join inflates the input to those operations without contributing to their result. Pushing the join later, after the expensive operations have reduced the data,

avoids carrying the join product through them. As an additional pattern, prune analysis also flags unused projected columns whose elimination simplifies upstream definitions.

What distinguishes prune analysis from local dead-code elimination in a single-query optimizer is its scope: a redundant or mistimed join in one model is invisible to a single-query rewriter, which has no view of how downstream models will use the result. Each detected candidate is scored by the cost of the affected operation, weighted by the number of downstream models whose simplified definitions benefit from the change.

From candidates to subgraphs. The two analyses produce a ranked stream of candidate model groups. From each group, a subgraph of the DAG is extracted that includes the candidate models together with a configurable neighborhood of upstream and downstream models, so that the LLM has enough surrounding context to reason about correctness. Subgraph size is bounded to keep the LLM input manageable; groups whose neighborhoods exceed the bound are skipped. The next subsection describes how the LLM consumes these subgraphs and produces concrete refactorings.

3.2 LLM Refactoring

Asking an LLM to rewrite SQL that will be deployed against a real warehouse raises three problems. The proposed rewrite may run *slower* than the original, may be *incorrect*, or may be *too expensive to produce naively* when a candidate region contains many models. DAGSmith addresses these with a four-stage sequence: subgraph analysis and opportunity proposal, proposal filtering, refactoring, and equivalence verification. A separate, lighter mechanism handles batches of near-duplicate models and is described at the end of the section.

Subgraph analysis and opportunity proposal. The first LLM call ingests the subgraph’s SQL and dependency graph and produces a domain-level reading of the region: a short semantic summary of each model and an account of the end-to-end business question the subgraph answers. This domain-level read is what allows subsequent stages to recognize semantically equivalent computations expressed in syntactically dissimilar SQL, the capability that distinguishes DAGSmith from purely syntactic rewriters. The headline

output is a list of *optimization opportunities*: each names the CPU-bound computation it would eliminate or shrink and the mechanism by which it would do so. Opportunities are not yet rewrites; they are proposals to be reviewed before any SQL is generated.

Proposal filtering. A second LLM call acts as an adversarial critic over the proposed opportunities. The separation is intentional: a single LLM asked simultaneously for proposals and for skepticism produces neither, with proposal mode dominating. A separate pass instructed to assume the proposer is overconfident behaves differently. Filtering at this point also saves cost, since the refactoring stage that follows is the most token-expensive phase in the pipeline.

The critic looks for both slowdowns and correctness traps. The slowdown case rejects, for instance, proposals that look like reuse-driven savings but add a new write and scan without reducing downstream work. A correctness trap is illustrated in Figure 6: a parent table joins to a multi-tag child to compute per-item flags, and the proposal eliminates the join by reading the tag from a sibling that has been pre-deduplicated to one row per item. Doing so silently drops the secondary tags that the original join admitted. The critic returns one of three verdicts: *approve*, *reject*, or *conditional*. A conditional verdict means the proposal is net-positive only under a specific data property; that property is recorded as a constraint that the refactoring stage must respect.

Original: join admits all tags	Proposal: drop join, read tag
<pre> 1 SELECT i.item_id, 2 MAX(CASE WHEN t.tag='A' 3 THEN 1 ELSE 0 END) AS flag_a 4 FROM ((ref('items'))) i 5 JOIN ((ref('item_tags'))) t 6 USING (item_id) 7 GROUP BY i.item_id </pre>	<pre> 1 SELECT s.item_id, 2 CASE WHEN s.tag='A' 3 THEN 1 ELSE 0 END AS flag_a 4 FROM ((ref('items_staged'))) s 5 -- items_staged keeps one tag/item; 6 -- secondary tags are silently lost </pre>

Figure 6: Correctness trap rejected by the proposal critic. The original aggregates over all tags per item; the proposal reads from a sibling that has been deduplicated to one tag per item, and silently drops the rest.

Refactoring. The opportunities that survive filtering are passed, together with the analysis and the SQL of the affected nodes, to a third LLM call that produces concrete refactored SQL. The LLM may rewrite existing nodes, mark nodes as removed, or introduce new shared nodes. Conditional opportunities from the previous stage carry their constraints into this prompt. Nodes with consumers outside the subgraph may have their internal SQL rewritten but must produce identical output rows, since their contract with downstream consumers is fixed.

Equivalence verification. A rewrite that survives filtering may still silently change the data. A bug rejected at the proposal stage can re-emerge inside a more complex rewrite that bundles several opportunities together: the same staged-input shortcut from Figure 6, for instance, can reappear nested inside a consolidated aggregation that the proposal critic approved on other grounds. DAGSmith treats equivalence as an empirical question against the pipeline’s real data. A fourth LLM call, the *correctness critic*, examines the original-versus-refactored SQL of every rewritten node whose output is externally visible and, for each adopted opportunity, articulates the data invariant that must hold for the rewrite to

be semantically safe. Each invariant is rendered as a SQL diagnostic, written so that it returns zero rows when the invariant holds and rows when it is violated. The diagnostics are executed against the target warehouse with a per-query byte cap. A non-empty result is a counterexample drawn from real data: the rewrite changes output. Counterexamples re-enter the pipeline as feedback. The failed invariants are appended to the refactoring prompt, and the LLM is asked to either propose a distinctly different strategy that respects them or restore the original logic. The loop continues until the refactor passes all diagnostics, the LLM declares no further distinct strategies, or a per-group attempt budget is exhausted, in which case the group is reverted to its original SQL and contributes no candidate.

Model families and propagation. A single subgraph can contain many near-duplicate models that perform the same operation over different data subsets. The subgraph analysis identifies such groups, designates a single *representative* per group, and records how the other members differ. Only the representative is taken through the four-stage sequence above; once it passes verification, the change is applied to the remaining members by a separate, lighter LLM call per member, executed in parallel. This is a scalability mechanism specific to projects whose models exhibit family structure; in projects without it, the family map collapses to singletons and the main sequence runs over every model directly.

3.3 Materialization Tuning

A candidate refactoring does not have a fixed cost: whether it improves the DAG depends on which of its nodes are materialized as tables and which are inlined as views, and the right answer differs from one candidate to the next. DAGSmith therefore evaluates every candidate under its own optimal materialization assignment, computed by the procedure described in this subsection. The original DAG, with no refactoring applied, is itself one of these candidates and serves as the baseline against which the others are scored.

Cost of a DAG configuration. Given a materialization assignment, the cost of the pipeline is computed by a single topological pass over the DAG. A table incurs its own predicted slot time once per execution and exposes its predicted cardinality to its children; a view incurs no slot time of its own and instead pushes its SQL features and its parents’ cardinality forward into every node that consumes it, scaled by the number of times it is referenced. The features a node is scored on are therefore not the raw counts extracted from its own SQL alone but a vector that has accumulated the contributions of every view in its upstream chain. The cost a node pays consequently depends on the materialization of each of its ancestors, and flipping a single node from table to view can change the predicted cost of every descendant.

Cost model. Two Huber regression models, trained on the history of instrumented runs of the original pipeline, predict the per-node quantities consumed by the topological pass. The first predicts the cardinality of each materialized table from a set of SQL features extracted by parsing each node’s compiled query: counts of joins, equi- and non-equi join keys, predicates broken down by kind (equality, range, IN), group-by clauses and keys, distinct operations, unions, and table scans. The second predicts slot time from a closely related set of features that adds window functions, total aggregations, and

order-by clauses. Both predictions are conditioned on the cardinality of the node’s parents under the current configuration. Both training and prediction operate on view-expanded feature vectors, so the regression sees, for each node, exactly the work it would actually perform under the configuration in question. Huber loss is used in place of squared error to limit the influence of a small number of very large queries.

Iterative local linearization. The total cost is a discrete non-linear function of the materialization vector $t \in \{0, 1\}^{|V|}$ with interactions that propagate through every view chain in the DAG; exact minimization is intractable for pipelines of realistic size. DAGSmith therefore applies iterative local linearization, a standard successive-approximation scheme for discrete non-linearities: linearize the cost around the current configuration, solve the linear surrogate, move, and repeat until the configuration stops changing. The procedure converges to a locally consistent assignment rather than a global optimum, which is the practical price of feasibility.

Concretely, let $t_i \in \{0, 1\}$ be the materialization of node i (table if $t_i = 1$, view if $t_i = 0$). At iteration k , with the current assignment $t^{(k)}$ fixed, DAGSmith measures two local quantities using the cost model: the baseline build cost $B_j^{(k)}$ of every node j under $t^{(k)}$ with its direct parents forced to tables, and the edge-local delta $U_{ij}^{(k)}$, the additional cost incurred at j when only parent i is flipped to a view (other parents held as tables). Treating $B^{(k)}$ and $U^{(k)}$ as constants for the duration of the iteration, the system solves a small ILP for the next assignment $t^{(k+1)}$:

$$\begin{aligned} \min_{t, y} \quad & \sum_{j \in V} t_j B_j^{(k)} + \sum_{i \rightarrow j \in E} y_{ij} U_{ij}^{(k)}, \\ \text{s.t.} \quad & y_{ij} \leq 1 - t_i, \quad y_{ij} \leq t_j, \quad y_{ij} \geq t_j - t_i, \quad y_{ij} \geq 0, \end{aligned}$$

where the auxiliary $y_{ij} \in [0, 1]$ is constrained to take value $(1 - t_i) \wedge t_j$: it is 1 precisely when j executes and reads from a parent i that has been inlined as a view, and 0 otherwise. A node j thus pays its baseline $B_j^{(k)}$ when materialized as a table and an additional $U_{ij}^{(k)}$ for each parent that is a view. To keep each step inside the region where the local coefficients remain valid, the ILP is constrained by a trust-region cap on the number of flips per iteration and a small flip penalty that damps oscillation. The new assignment becomes the current one and the loop repeats; B and U are recomputed because flipping a node changes the predicted cardinalities seen by its descendants. The loop terminates on convergence, on revisiting a previously seen assignment, or after a fixed iteration cap. Two pieces of domain knowledge keep the search well behaved: nodes with many descendants are pinned as tables, since inlining them would cause their work to be redundantly recomputed for every consumer, and seed and source tables are fixed as tables throughout, since their materialization is not under the project’s control.

From a candidate to a delta. For the original project, the procedure above runs directly: every model has a measured cardinality and slot time from the instrumented run, and the loop searches the materialization assignment that minimizes total cost. For a refactored candidate, models that did not exist in the original have no measurements. The candidate’s compiled manifest is fed through the same topological pass, with the cost model invoked once per new model to predict its cardinality from its SQL features and the (predicted or measured) cardinality of its parents; from that point

the candidate is indistinguishable from the original as far as the optimizer is concerned, and the same iterative loop produces its optimal materialization. The cost of the candidate under that materialization, subtracted from the cost of the original under its own, is the candidate’s *delta*: the predicted cost reduction the refactoring delivers when adopted in isolation. The next subsection composes the candidates given these deltas.

3.4 Global Selection

The previous stage produces, for each candidate refactoring, a delta computed as if that candidate were adopted alone. Composing the candidates into a single optimized project is not a matter of taking them all: the structural analyses of Section 3.1 surface overlapping regions of the DAG, so the same model can belong to more than one group, and the LLM has rewritten each group in isolation; applying two refactorings that touch the same model would require a merge that no stage of the pipeline has produced and whose correctness no stage has verified. A separate constraint comes from *within* each group, where the LLM may have proposed several variants of the same refactoring opportunity, of which at most one can be adopted. Global selection picks the subset of candidates that maximizes total delta subject to both restrictions.

Conflict graph. Two groups are in conflict when they touch (rewrite, create, or remove) any of the same models. DAGSmith computes a conflict graph by intersecting the changed-model sets of every pair of groups; an edge is added whenever the intersection is non-empty. The graph is typically sparse, since the structural analyses tend to surface regions whose dataflow patterns differ, but it is rarely empty: shared upstream models that participate in multiple opportunities are the common source of edges.

ILP selection. Let $x_c \in \{0, 1\}$ indicate whether candidate c is adopted, with delta δ_c , and let $z_g \in \{0, 1\}$ indicate whether any variant from group g is adopted. DAGSmith solves

$$\max_{x, z} \quad \sum_c \delta_c x_c \quad \text{s.t.} \quad \sum_{c \in g} x_c \leq z_g \quad \forall g, \quad z_g + z_{g'} \leq 1 \quad \forall (g, g') \in C,$$

where C is the edge set of the conflict graph. The first constraint enforces that at most one variant per group is adopted; the second enforces that no two conflicting groups are adopted together. The problem is small (one binary per candidate and per group, with a sparse constraint matrix) and is solved exactly by an off-the-shelf MILP solver. Once the subset is fixed, the materialization tuning of Section 3.3 is run once more on the combined project, since each candidate’s delta was measured under its own isolated materialization assignment and the optimal assignment for their combination need not coincide with any one of them.

3.5 Frequency-Aware Optimization

The previous stages treated every node’s slot time as a fixed per-execution cost. In a recurring SQL pipeline DAG this is an oversimplification: different sources update at different rates, different downstream consumers require fresh output at different rates, and the cost a node contributes to the pipeline is the cost of one execution multiplied by how often it actually executes. DAGSmith therefore augments the pipeline with a frequency dimension that enters at three layers. Frequencies are taken from the declared

schedule at sink nodes and from observed update rates at source tables, and propagated through the interior of the DAG from these boundaries. Layers 1 and 2 reweight and reschedule the existing DAG without changing its structure; layer 3 contributes a new class of structural candidates back to the candidate generator of Section 3.1.

Layer 1: Frequency as cost weight. The materialization tuning of Section 3.3 and the selection of Section 3.4 both minimize total cost as if every model executed at the same rate. Replacing per-execution slot time with frequency-weighted slot time changes the answer: a refactoring that saves work on a daily-refresh model is worth more than the same per-execution savings on a weekly one. Mechanically, the local coefficients of the iterative linearization are reweighted before the ILP runs:

$$B_j^{(k)} \leftarrow f_j \cdot B_j^{(k)}, \quad U_{ij}^{(k)} \leftarrow f_j \cdot U_{ij}^{(k)},$$

where f_j is node j 's effective frequency. The rest of Section 3.3 is unchanged, and the selection ILP inherits the same reweighting through the deltas it consumes.

Layer 2: Reconciling scheduled and effective frequency. A model's effective frequency is not the rate at which its schedule fires; it is the rate at which its schedule fires *and* produces new output. Each model has a *demand* frequency f_j^{demand} , set by the rate at which its downstream consumers need fresh output, and a *data-changing* frequency f_j^{data} , set by the rate at which any of its upstream inputs actually change. The rate at which the model contributes useful work is

$$f_j = \min(f_j^{\text{demand}}, f_j^{\text{data}}).$$

Where the scheduled rate exceeds f_j , the model is being over-scheduled: it executes, reads its inputs, and produces output identical to what it produced last time. DAGSmith flags such models and reduces their schedule to f_j . This layer changes no SQL and no materialization, only the schedule, but the saving is immediate: every redundant execution is eliminated outright. The same f_j is what layer 1 uses as the cost weight, so the two layers compose.

Layer 3: Frequency-driven splitting hints. The first two layers leave the DAG structure intact. The third generates new structural candidates of a kind that the analyses of Section 3.1 cannot identify on dataflow shape alone. When a model has parents with different data-changing frequencies, it inherits the fastest one; any work inside the model that depends only on slow-changing parents is therefore re-executed at the fast rate even though its inputs have not changed. DAGSmith identifies models whose parent frequencies are heterogeneous and whose slow-side computation is non-trivial, and emits the location to the candidate generator as a hint, alongside the reuse and prune analyses of Section 3.1. The hint enters the LLM stage in the same form as any other candidate region; the LLM proposes a frequency-split refactoring of the kind illustrated in motivating Example 3, in which the slow-side computation is extracted into a separate model that runs on the slow cadence and the original model becomes a lightweight join over its outputs. From there the candidate flows through the same filtering, refactoring, verification, scoring, and selection stages as any other, with layers 1 and 2 active throughout.

4 EVALUATION PLAN

This draft currently focuses on problem formulation and system design. The final version should evaluate DAGSmith along four questions: whether it reduces end-to-end warehouse cost, which classes of dependency-aware refactorings account for the gains, how often the verification loop rejects unsafe rewrites, and how the benefits vary with workload frequency skew and DAG size.

4.1 Workloads and Baselines

The evaluation should include at least one real dbt project, one SQLMesh- or warehouse-task-style pipeline when available, and semi-synthetic benchmarks derived from recurring analytics pipelines. Natural baselines include the original DAG, materialization-only tuning, schedule-only tuning, structural analyses without LLM refactoring, and single-query rewrite systems that do not cross node boundaries.

4.2 Metrics

Primary metrics should include total slot time or warehouse cost over a fixed horizon, refresh latency at the DAG sinks, the number of nodes changed, and verification pass rate. Secondary diagnostics should measure optimizer runtime, LLM usage, and the distribution of wins across semantic reuse, contract-aware rewriting, computation placement, materialization coupling, and freshness-driven restructuring.

4.3 Ablations

The final paper should isolate the contribution of each layer: candidate identification, proposal filtering, equivalence diagnostics, materialization tuning, and frequency-aware reasoning. A particularly important ablation is whether frequency-aware splitting produces gains that materialization-only optimization cannot recover.

5 RELATED WORK

The most closely related lines of work are database query optimization, source-to-source SQL rewriting, multi-query optimization, materialized-view recommendation, workflow scheduling, and data lineage or clone analysis for analytics pipelines. DAGSmith differs from these lines by treating the explicit SQL pipeline DAG itself as the optimization object, allowing it to restructure node boundaries and reason jointly about semantics, materialization, and refresh frequency.

Database query optimizers remain essential but orthogonal: they choose physical plans for one submitted SQL statement. DAGSmith operates before that point, changing the maintained SQL pipeline program that produces those statements. Source-to-source SQL rewriters such as *SlabCity* [?], *GenRewrite* [?], *WeTune* [?], and *R-Bot* [?] also operate before the database optimizer, but they optimize one statement at a time and do not change the dependency graph around it.

Multi-query optimization and common-subexpression sharing reason across multiple statements, but typically over flat workloads and mostly through syntactic overlap. They do not usually exploit output contracts, declared refresh cadences, materialization choices, or stable developer-maintained node boundaries. Materialized-view selection optimizes persistence decisions, but does not usually

rewrite the graph to move computation across freshness boundaries or remove intermediate contracts. Workflow systems such as Airflow [?], Dagster [?], Prefect [?], Argo [?], Luigi [?], SQLMesh [?], and Databricks Workflows [?] expose dependencies and schedules, but their optimizers primarily decide ordering, invalidation, or execution policy rather than semantic SQL rewrites. DAGSmith targets the gap between these areas: dependency-aware source-to-source optimization for recurring SQL pipeline DAGs.

6 CONCLUSION

Explicit SQL pipeline DAGs create an optimization setting that is richer than traditional one-query-at-a-time rewriting. Once dependencies, node contracts, materialization choices, and refresh

behavior are visible, the optimizer can restructure the pipeline itself rather than merely improve individual statements. DAGSmith is a step toward that goal: it treats dependency-aware pipeline optimization as a joint problem over graph structure, materialization, and frequency, and combines structural analysis with LLM-guided refactoring and verification to explore that space.

The core claim is that recurring SQL pipeline DAGs expose whole-pipeline optimization opportunities that are invisible to query-local rewriting, and exploiting them requires a system designed for dependency-aware reasoning.